

YOLOv11 Object Tracking

YOLOv11 Object Tracking

1. Course Content
2. Launch Case
3. Parameter Tuning Guide
 - 3.1 Temporary Parameter Tuning
 - 3.2 Long-term Effective Parameters
3. Core Code Analysis

1. Course Content

- Run examples, use YOLOv11 model to detect targets and mark unique tracking IDs for targets, the car will follow the target to move

[!IMPORTANT]

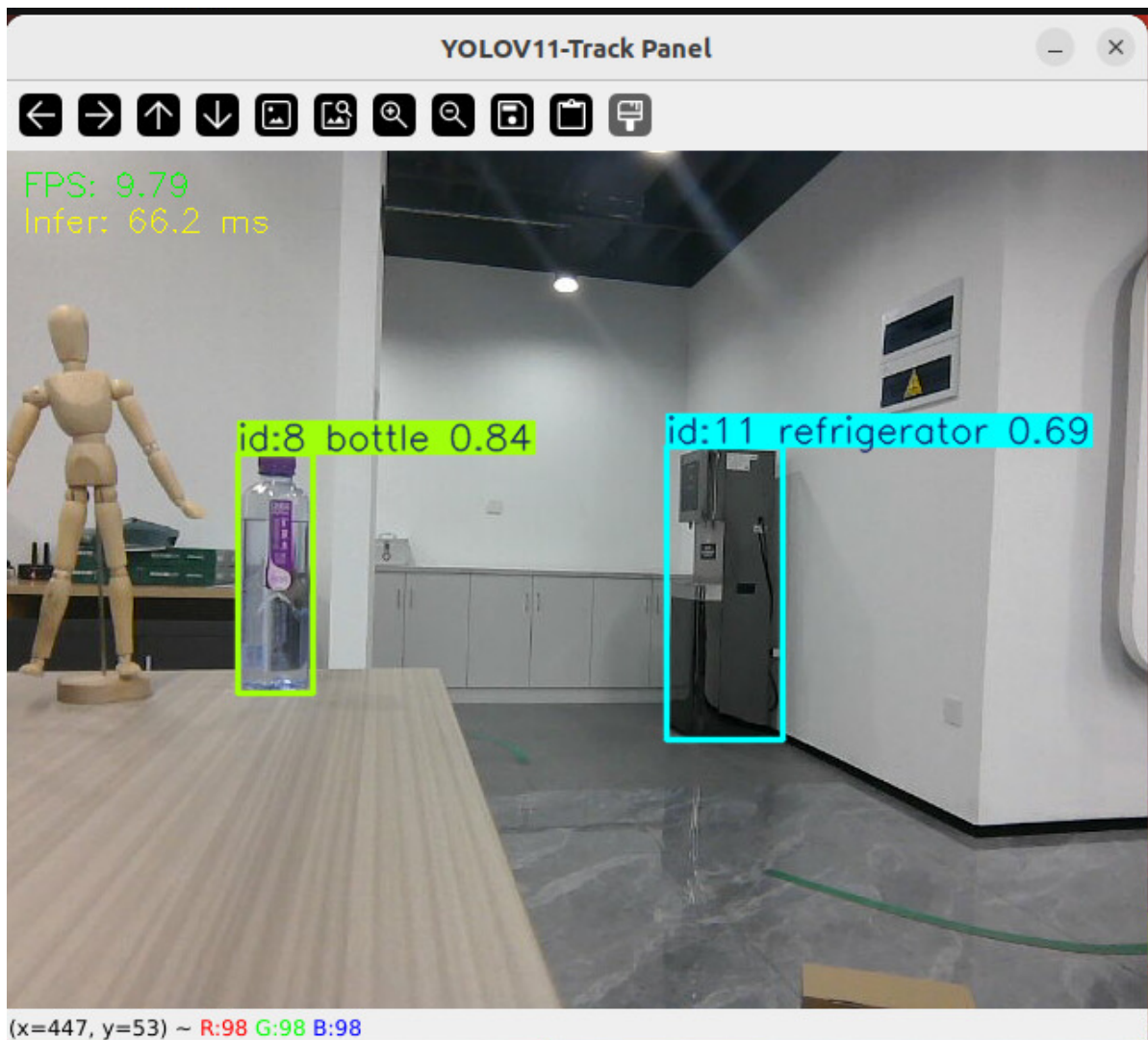
Due to performance and library-related issues, this case is exclusive to ORIN mainboard!!

2. Launch Case

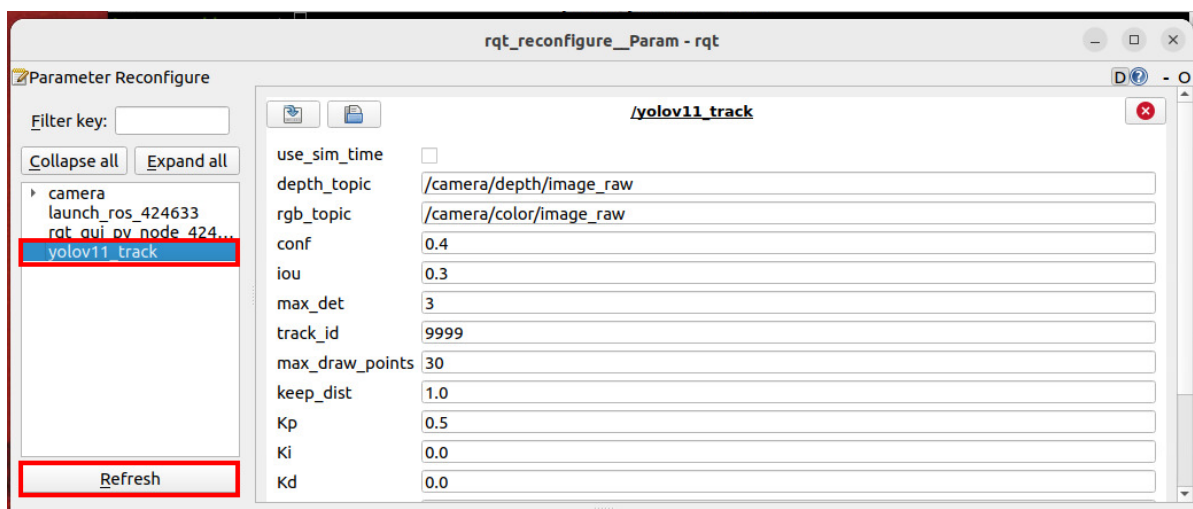
- Run command in terminal:

```
ros2 launch m3_bringup depthcam_demo.launch.py demo:=yolov11_track
```

After the program runs, a `YOLOv11-Track Panel` window opens, performing object detection through the YOLOv11 model and marking each target with a unique ID, where FPS is the average frame rate per second, and Infer is the inference time for each frame image

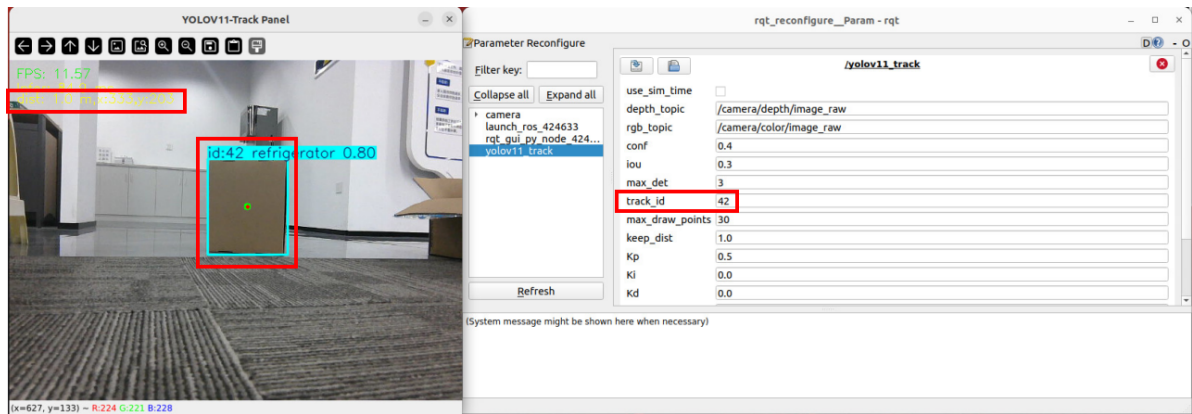


- After the program starts, it will also automatically start the `rqt_reconfigure` parameter configuration panel. Click the `yolov11_track` option on the left. If it doesn't refresh initially, click `Refresh` to refresh



- Fill in the `track_id` with the target to be tracked, the car will follow the target in the frame and move, maintaining a `keep_dist` distance from the target, and placing the target in the left-right center position of the frame.
- The car will mark the target and display the target's center point and movement trajectory (by default saves the latest 30 center point trajectories), while the left side will show the

distance `dist` from the target center point, and the coordinates (x,y) of the target center point,



- The x and y axis directions of the frame are defined as follows, frame size is: 640*480



3. Parameter Tuning Guide

[!TIP]

- Parameter tuning can achieve better object tracking performance

3.1 Temporary Parameter Tuning

All adjustable parameters are as follows. In `rqt_reconfigure`, you can temporarily tune parameters. If you need to make tuned parameters effective long-term, you need to write the parameters to the parameter configuration file:

yolov11_track	
conf	0.4
iou	0.3
max_det	3
track_id	42
max_draw_points	30
keep_dist	1.0
Kp	0.5
Ki	0.0
Kd	0.0
Kp_1	-0.1
Ki_1	0.0
Kd_1	-0.3
max_integral	1000.0

Parameter functions:

- **conf** Confidence, set the minimum confidence threshold for detection. Detected objects with confidence below this threshold will be ignored. Adjusting this value helps reduce false positives
- **iou** Overlap threshold used for Non-Maximum Suppression (NMS). Lower values reduce detection results by eliminating overlapping boxes
- **max_det** Maximum number of objects to detect
- **max_draw_points** Maximum number of target center point trajectories to retain
- **keep_dist** Distance from target
- **Kp** Kp for distance following
- **Ki** Ki for distance following
- **Kd** Kd for distance following
- **Kp_1** Kp for target left-right following centering
- **Ki_1** Ki for target left-right following centering
- **Kd_1** Kd for target left-right following centering
- **max_integral** Kd for target following and centering

3.2 Long-term Effective Parameters

- If you need long-term effective parameters, open the following parameter file:

```
~/yahboomM3_ws/user_ws/src/m3_bringup/config/common_param.yaml
```

- Fill in the tuned parameters into corresponding fields to make parameters effective long-term

```
yolov11_track:
  ros__parameters:
    conf: 0.4

    iou: 0.3
    max_det: 3
    max_draw_points: 30
    keep_dist: 1.0
    Kp: 0.5

    Ki: 0.0
    Kd: 0.0
    Kp_1: -0.1
    Ki_1: 0.0
```

```
kd_1: -0.3
max_integral: 1000.0
```

- The object recognition types here are the same as those in the previous YOLOv11 object detection. To get the highest frame rate and inference speed, the YOLOv11 model used in this case has been optimized for different platforms (OrinNano, OrinNX, Raspberry Pi, RDKx5) in model format, model paths are as follows:

```
~/MODELS/yo1ov11/
```

- Program source code paths are as follows:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/yolov11/yolov11_track.py
```

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/yolov11/yolov11_infer.p
y
```

##

- Class in yolov11_infer.py: YOLO_Track is a core functionality program that encapsulates model loading, inference, and target ID detection

```
class YOLO_Track(YOLO_MODEL):  
    """YOLOv11 object tracking"""  
  
    def __init__(self,model_path:str):  
        self.model_path=model_path  
  
    def init_yolo_engine(self):  
        "Initialization of model"  
        try:  
            self.yolo_model = YOLO( self.model_path,task="detect")  
        except:  
            return False  
        return True  
  
    def get_infer_result(self,source,conf,iou,max_det,stream:bool=True):  
        '''Obtain the reasoning result of a single frame image'''  
        result = self.yolo_model.track(source,  
                                       conf=conf,  
                                       iou=iou,  
                                       half=True,  
                                       stream=stream,  
                                       max_det=max_det,  
                                       persist=True,  
                                       verbose=False)  
  
        return result
```

```
def check_target_id(self, target_id:int16, track_ids, boxes):
    '''Check if target ID exists'''
    for box, track_id in zip(boxes, track_ids):
        if track_id == target_id:
            return box
    return None
```

- The depth_img_Callback callback function in yolov11_track.py program is the core program for displaying detection frames, with the following functions:
- Subscribe to camera topic
- Use yolo_model.get_infer_result function method to perform inference on single frame images of camera feed and draw on the frame, here tracking mode is used
- Use cv2.imshow function to display image frame rate and inference time

```
def depth_img_Callback(self, rgb_msg:Image, depth_frame:Image):
    '''RGB and depth image callback function'''
    box=None
    current_pose=None
    # convert image
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(rgb_msg,
desired_encoding='bgr8')
    depth_image = self.depth_bridge.imgmsg_to_cv2(depth_frame,
self.encoding[1])
    depth_image = cv2.resize(depth_image, (640, 480))
    depth_image_info = depth_image.astype(np.float32)
    infer_start = time.time()
    #yolo infer
    results=
self.yolo_model.get_infer_result(rgb_image, conf=self.conf, iou=self.iou, max_det=s
elf.max_det, stream=True)
    for res in results:
        frame = res.plot()
        if res.bboxes and res.bboxes.is_track:
            boxes = res.bboxes.xywh.cpu() # (x,y,w,h)
            track_ids = res.bboxes.id.int().cpu().tolist()

box=self.yolo_model.check_target_id(self.track_id, track_ids, boxes)
            if box is not None:
                self.draw_trajectory_point(frame, box)
                current_pose=self.get_target_dist(box, depth_image_info)
                if current_pose is not None :
self.move_control(current_pose[0], current_pose[1])
                break
            #handle result
            if current_pose is not None:
                cv2.putText(frame, f"dist: {current_pose[0]:.1f} m, x:
{current_pose[1]}, y:{current_pose[2]}", (10, 60), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,
255, 255), 1)
            else:
                '''stop move'''
                self.pub_count+=1
                if self.pub_count>=3:
                    self.pub_count=0
```

```

        self.cmd_vel_pub.publish(Twist())

    infer_end = time.time()
    self.frame_count += 1
    curr_time = time.time()
    time_diff = curr_time - self.prev_time
    if time_diff >= 1.0:
        self.fps = self.frame_count / time_diff
        self.frame_count = 0
        self.prev_time = curr_time
    cv2.putText(frame, f"FPS: {self.fps:.2f}", (10,
25), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 1)
    infer_time = (infer_end - infer_start) * 1000
    cv2.putText(frame, f"Infer: {infer_time:.1f} ms", (10,
45), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 1)
    cv2.imshow("YOLOV11-Track Panel", frame)
    cv2.waitKey(1)

```

- The following functions are for getting the distance of the target center point and drawing the target center point movement trajectory

```

def draw_trajectory_point(self, frame, box):
    '''Draw the trajectory and the center point'''
    x, y, w, h = box
    center_x, center_y = int(x), int(y)
    cv2.circle(frame, (center_x, center_y), radius=5, color=(0, 255, 0),
thickness=-1) #draw center point

    track = self.track_history[self.track_id] # get history of track
    track.append((float(x), float(y))) # add to history
    if len(track) > self.max_draw_points:
        track.pop(0)
    points = np.hstack(track).astype(np.int32).reshape((-1, 1, 2))
    cv2.polylines(frame, [points], isClosed=False, color=(0, 0, 255),
thickness=3)

def get_target_dist(self, box, depth_image_info):
    '''Get the target distance'''
    x, y, w, h = box
    valid_depths = []
    center_x, center_y = int(x), int(y)
    points = [
        (int(center_y), int(center_x)),
        (int(center_y - 1), int(center_x - 1)),
        (int(center_y - 3), int(center_x - 3)) ]
    for y, x in points:
        depth_val = depth_image_info[y][x] #Read the depth
value corresponding to the coordinates in the depth map
        if depth_val != 0: valid_depths.append(depth_val)

    if valid_depths:
        dist = (sum(valid_depths) / len(valid_depths))/1000 #convert to
meter
        return dist, x, y

```

return None